

CS3160 — Simulator Code Map

Every file in `src/`, what is in it, and the lab you write it in. A file belonging to a later lab is not in your folder at all — what you are given always builds and runs on its own. Every place you fill in is marked `TODO: grep -rn TODO src/` is this lab's work.

The files

File	What's in it	Lab	Key types
<code>common.h</code>	u32, i32 and friends; <code>sign_extend()</code> , <code>bits()</code> , <code>fatal()</code>	given	—
<code>log.{h,cpp}</code>	<code>log_out()</code> — the one [OUT] line per retired instruction	given	<code>LogLevel</code>
<code>config.{h,cpp}</code>	reads and checks the <code>-config</code> file	given	<code>Config</code> , <code>CacheConfig</code>
<code>stats.{h,cpp}</code>	the counters, and <code>stats.json</code>	given†	<code>Stats</code> , <code>LevelStats</code>
<code>memory.h</code>	the four calls every memory level answers	given	<code>MemoryLevel</code>
<code>ram.{h,cpp}</code>	byte-addressed memory, and <code>-dump</code>	lab 3	<code>Ram</code>
<code>loader.{h,cpp}</code>	the <code>.r50b</code> image into RAM	lab 3	—
<code>decode.{h,cpp}</code>	32 bits into a <code>Decoded</code> ; <code>is_branch()</code> , <code>is_load()</code> , ...	lab 4	<code>Decoded</code> , <code>Op</code> , <code>Format</code>
<code>disasm.{h,cpp}</code>	a <code>Decoded</code> into the text <code>objdump</code> prints	lab 4	—
<code>alu.{h,cpp}</code>	what the arithmetic does, and when a branch is taken	lab 5	<code>AluInputs</code>
<code>processor.{h,cpp}</code>	the registers, and the stage functions every model shares	lab 5	<code>Processor</code> , <code>Fetch</code> , <code>Executed</code> , <code>MemAccess</code> , <code>Latch</code>
<code>processor_single.cpp</code>	<code>run_single()</code> — one instruction, one cycle	lab 5	—
<code>processor_pipelined.cpp</code>	<code>run_pipelined()</code> — five stages, four latches, the flush	lab 6	<code>Latch</code>
<code>hazard.{h,cpp}</code>	which instruction depends on which, and what to do about it	labs 6–8	<code>Hazard</code> , <code>Forwarding</code> , <code>FwdSrc</code>
<code>processor_forwarding.cpp</code>	<code>run_forwarding()</code> — operands taken from where they are	lab 8	—
<code>cache.{h,cpp}</code>	one cache level: lookup, replacement, the write path	labs 9–10	<code>Cache</code> , <code>CacheBlock</code> , <code>CacheAddr</code>
<code>sim.cpp</code>	<code>main()</code> : the options, the machine it builds, the run	given	—

† `stats.cpp` is given except `level_amat()`, which is lab 10. — The three `processor_*.cpp` are three separate run loops on purpose: three machines running the same instructions, so you can put their `stats.json` side by side.

What you write, lab by lab

Lab	File	Functions
3	<code>ram.cpp</code> , <code>loader.cpp</code>	<code>ram_read</code> <code>ram_write</code> <code>ram_dump</code> <code>loader_load</code>
4	<code>decode.cpp</code> , <code>disasm.cpp</code>	<code>decode</code> <code>disasm</code>
5	<code>alu.cpp</code> , <code>processor.cpp</code> , <code>processor_single.cpp</code>	<code>alu_apply</code> <code>branch_taken</code> <code>stage_execute</code> <code>next_pc_of</code> <code>stage_memory</code> <code>stage_writeback</code> <code>run_single</code>
6	<code>processor_pipelined.cpp</code> , <code>hazard.cpp</code>	the five stages, the flush and the commit inside <code>run_pipelined</code> ; <code>redirects_pc</code> <code>wrong_path_instructions</code>
7	<code>hazard.cpp</code>	<code>hazard_stalling</code>
8	<code>processor_forwarding.cpp</code> , <code>hazard.cpp</code>	the EX and ID halves of <code>run_forwarding</code> ; <code>hazard_forwarding</code> <code>forward_in_ex</code> <code>forwarded_value</code>
9	<code>cache.cpp</code>	the geometry switch in <code>cache_init</code> ; <code>cache_split</code> <code>cache_block_addr</code> <code>cache_find</code> <code>cache_order_insert</code> <code>cache_order_touch</code> <code>cache_choose_way</code> <code>cache_evict_for</code> <code>cache_fill</code> <code>Cache::read</code>
10	<code>cache.cpp</code> , <code>stats.cpp</code>	<code>cache_writeback_if_dirty</code> <code>Cache::write</code> <code>Cache::read_block</code> <code>Cache::write_block</code> <code>cache_flush</code> <code>level_amat</code>

The structs, in the order one instruction meets them

<code>Fetch</code>	<code>raw</code> , <code>pc</code> , <code>latency</code>	what came back from <code>imem</code>
<code>Decoded</code>	<code>op</code> , <code>fmt</code> , <code>rs1</code> , <code>rs2</code> , <code>rd</code> , <code>imm</code>	what those 32 bits mean
<code>Executed</code>	<code>result</code> , <code>taken</code>	the ALU's answer; <code>result</code> is a value, an address, or a branch target
<code>MemAccess</code>	<code>did_read</code> , <code>did_write</code> , <code>addr</code> , <code>value</code> , <code>latency</code>	what the load or store did
<code>Latch</code>	<code>valid</code> , <code>d</code> , <code>v_rs1</code> , <code>v_rs2</code> , <code>e</code> , <code>next_pc</code> , <code>m</code>	all of the above, for one instruction in flight -- lab 6 onwards

`Processor` holds what survives an instruction: `pc`, `regs[32]`, and the `imem` and `dmem` it reads through. `Stats` holds what you are measured on. Both are given to you in lab 5, and neither struct changes for the rest of the course.

Where the memory hierarchy plugs in

`MemoryLevel` is the interface: `read`, `write`, `read_block`, `write_block`. `Ram` implements it, and from lab 9 `Cache` implements it too and holds a `next` pointer to the level below. That is the whole trick — the processor calls `imem->read()` without knowing whether a cache is on the other side, so nothing you wrote in labs 3–8 changes when you add one.

```
cpu.imem -> I1 -> RAM      an instruction cache
cpu.dmem -> L1 -> L2 -> RAM a two-level data side
```